МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №2

по курсу «Программирование графических процессоров»

Обработка изображений на GPU. Фильтры

Выполнил: Л.А. Фокин

Группа: М8О-401Б-23

Преподаватель: А.Ю. Морозов

Москва, 2026

Содержание

1 Условие **2**
2 Программное и аппаратное обеспечение **2**
3 Метод решения **3**
4 Описание программы **5**
5 Результаты **9**
6 Выводы **10**

1 Условие

Цель работы и общая постановка задачи

Целью лабораторной работы является освоение обработки изображений на GPU с использованием текстурной памяти и двухмерной сетки потоков, а также практическая реализация фильтра изображения с оценкой достигнутого ускорения относительно CPU.

Вариант задания

Вариант 1. Гауссово размытие. Необходимо реализовать размытие по Гауссу с линейной сложностью от радиуса размытия (двухпроходный алгоритм).

- **Входные данные:** на первой строке задается путь к исходному изображению, на второй — путь к конечному изображению. На следующей строке задано целое число r — радиус размытия ($w \cdot h \leq 5 \cdot 10^7$, $0 \leq r < 1024$).
- **Выходные данные:** обработанное изображение, записанное в тот же бинарный формат: первые восемь байт — размеры w и h (тип `int`), далее построчно значения пикселей r, g, b, a (по 4 байта на пиксель).
- **Ограничения:** используются только цветовые составляющие (r, g, b), альфа-канал не учитывается; допускается ошибка в ± 1. В пограничном случае изображение "расширяется" за границы дублированием граничных пикселей: $ip := \max(\min(i, h), 1)$, $jp := \max(\min(j, w), 1)$.

2 Программное и аппаратное обеспечение

Аппаратное обеспечение

- **Графический процессор (GPU):** NVIDIA GeForce RTX 5070
  - Compute Capability: 12.0
  - Объем глобальной памяти (Global Memory): 12 792 МБ ($\approx$ 12 ГБ) GDDR7
  - Количество потоковых мультипроцессоров (SM): 48
  - Максимальное количество нитей на блок: 1024
  - Максимальный размер блока: $(1024, 1024, 64)$
  - Максимальный размер сетки: $(2147483647, 65535, 65535)$
- **Центральный процессор (CPU):** AMD Ryzen 5 8400F 6-Core Processor (6 физических ядер, 12 логических потоков, базовая тактовая частота 4.2 ГГц, Max Boost до 4.7 ГГц, кэш L3 16 МБ)
- **Оперативная память (RAM):** 16 ГБ DDR5
- **Жесткий диск (Накопитель):** NVMe Solid State Drive (SSD) емкостью 1 ТБ, файловая система ext4

Программное обеспечение

- **Операционная система (OS):** Ubuntu 24.04 LTS (x86_64 Linux kernel 6.6)
- **Среда разработки (IDE / Редактор):** Zed Editor / Visual Studio Code
- **Компилятор CUDA:** NVIDIA CUDA Compiler Driver (nvcc) v12.0 (V12.0.140)
- **Компилятор C++ хоста:** g++ (Ubuntu 13.3.0)

3 Метод решения

Формат изображения и идиома Threaded 2D

Изображение хранится в бинарном формате: w, h (4-байтовые целые), далее построчно пиксели `uchar4` (r, g, b, a). Каждому пикселю сопоставляется своя нить GPU: идея **двухмерной сетки потоков** — глобальные индексы нити вычисляются как

```
int idx = blockIdx.x * blockDim.x + threadIdx.x; // x (column)
int idy = blockIdx.y * blockDim.y + threadIdx.y; // y (row)
```

Для обработки произвольных размеров при фиксированной сетке применяется двухмерный grid-stride цикл: нить стартует со своего глобального индекса (idx, idy) и сдвигается с шагом (blockDim.x · gridDim.x, blockDim.y · gridDim.y) по обеим осям.

Двухпроходный алгоритм (сепарабельность ядра)

Двумерная гауссова свертка вычисляется по формуле

$$G'(i,j) = \sum_{k=-r}^{r} \sum_{m=-r}^{r} w(k)\, w(m)\, G(i+k, j+m), \qquad w(k) = \frac{e^{-k^2/(2r^2)}}{\sum_{t=-r}^{r} e^{-t^2/(2r^2)}},$$

где G — исходная яркость пикселя. Поскольку ядро $W(k,m) = w(k)\, w(m)$ сепарабельно, свертка раскладывается на два одномерных прохода:

1. **Горизонтальный проход:** $T(i,j) = \sum_k w(k) G(i, j+k)$ (радиус r);
2. **Вертикальный проход:** $G'(i,j) = \sum_k w(k) T(i+k, j)$ (радиус r).

Сложность алгоритма составляет $O(w \cdot h \cdot r)$ вместо $O(w \cdot h \cdot r^2)$ для наивной двумерной свертки; при $r = 100$ это сокращение числа умножений в 100 раз (402 операций против 40401 на пиксель).

Текстурная память

Промежуточные чтения соседних пикселей выполняются через текстурную память (`cudaTextureOb` поверх 2D-массива `cudaArray`):

- **режим адресации** `cudaAddressModeClamp` аппаратно реализует требуемое методичкой правило «расширения» изображения за границы: при выходе индекса за $[0, w) \times [0, h)$ возвращается граничный пиксель ($ip := \max(\min(i, h), 1)$), что исключает ветвления в ядрах;
- **точечная фильтрация** (`cudaFilterModePoint`) и ненормированные координаты обеспечивают точное чтение пикселя по координатам $(j + 0{,}5;\ i + 0{,}5)$;

- текстурный кэш локализует чтения: в каждом проходе пиксель читается $(2r + 1)$ раз, и обращения соседних нитей многократно попадают в кэш.

Исходное изображение загружается в текстуру `uchar4` один раз на этапе чтения; результат горизонтального прохода (`float4`, во избежание накопления погрешностей округления) копируется во вторую текстуру перед вертикальным проходом.

Использованные источники

1. NVIDIA Corporation. *CUDA C++ Programming Guide (Release 12.x)*. URL: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.

2. Морозов А. Ю. *Курс лекций по программированию графических процессоров (CUDA)*: Лекция 2 «Программные возможности CUDA». МАИ, 2024.

3. Методические указания к лабораторным работам по курсу «Программирование графических процессоров»: *Лабораторная работа №2. Обработка изображений на GPU. Фильтры.* МАИ, 2024.

4 Описание программы

Программа декомпозирована на исходный модуль на языке CUDA C++ (`main.cu`), CPU-референс для верификации (`report/cpu_ref.cpp`) и сборочный конфигурационный файл (`Makefile`).

Основные типы данных и функции

- **Типы данных:**
  - `uchar4` — пиксель изображения (8-битные каналы r, g, b, a);
  - `float4` — промежуточное представление канала r, g, b с плавающей точкой (`float`), a не учитывается;
  - `cudaTextureObject_t`, `cudaArray_t`, `cudaChannelFormatDesc` — для работы с текстурной памятью;
  - `dim3` — конфигурация сетки и блока потоков.
- **Макрос `CSC`:** перехватывает ошибки функций CUDA API, печатает сообщение в поток `stderr` с указанием файла, строки и описания через `cudaGetErrorString`, после чего завершает работу программы.
- **Ядро `gauss_horizontal`:** `__global__`-функция, горизонтальный проход гауссовой свертки; чтение пикселей через текстурный объект `uchar4` в режиме Clamp, запись результата (каждому пикселю своя нить, 2D grid-stride) в массив `float4`;
- **Ядро `gauss_vertical`:** `__global__`-функция, вертикальный проход; чтение промежуточного представления через текстурный объект `float4` в режиме Clamp, запись финального изображения в `uchar4` с ограничением значений отрезком $[0, 255]$ и обнуленным альфа-каналом;
- **Функции `make_object_uchar4`, `make_object_float4`:** создают 2D-текстурный объект (`cudaMallocArray` + `cudaCreateTextureObject`) с режимом адресации `cudaAddressModeC` точечной фильтрацией и ненормированными координатами;
- **Функции `read_image`, `write_image`:** бинарный ввод/вывод изображения в формате ЛР;
- **Функция `main`:** читает пути к файлам и радиус r, строит нормализованное ядро $w(k)$ и загружает его в глобальную память GPU, загружает изображение, запускает два ядра, синхронизирует GPU (`cudaEventSynchronize`), копирует результат на хост и записывает выходной файл; замер времени производится средствами CUDA Events.

Исходный код программы (`main.cu`)

```
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <ctime>

#include <cuda_runtime.h>

#define CSC(call)                                                       \
do {                                                                    \
    cudaError_t res = call;                                             \
```

```
    if (res != cudaSuccess) {                                                \
        std::fprintf(stderr, "ERROR in %s:%d. Message: %s\n",                 \
                     __FILE__, __LINE__, cudaGetErrorString(res));           \
        std::exit(0);                                                        \
    }                                                                        \
} while (0)

// Normalized Gaussian kernel w[i] = exp(-i^2 / (2 r^2)), built on the host.
// Separable two-pass scheme: complexity O(w*h*r) instead of O(w*h*r^2).

#define blur_tex(__tex, __i, __j) tex2D<uchar4>((__tex), (__j) + 0.5f, (__i) + 0.5f)
#define tmp_tex(__tex, __i, __j) tex2D<float4>((__tex), (__j) + 0.5f, (__i) + 0.5f)

__global__ void gauss_horizontal(cudaTextureObject_t src, float4* dst,
                                 int w, int h, const float* wts, int r) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int idy = blockIdx.y * blockDim.y + threadIdx.y;
    int offx = blockDim.x * gridDim.x;
    int offy = blockDim.y * gridDim.y;

    for (int i = idy; i < h; i += offy) {
        for (int j = idx; j < w; j += offx) {
            float3 acc = make_float3(0.0f, 0.0f, 0.0f);
            for (int k = -r; k <= r; ++k) {
                float f = wts[k + r];
                uchar4 p = blur_tex(src, i, j + k); // texture clamp extends the image
    borders
                acc.x += f * p.x;
                acc.y += f * p.y;
                acc.z += f * p.z;
            }
            dst[i * w + j] = make_float4(acc.x, acc.y, acc.z, 0.0f);
        }
    }
}

__global__ void gauss_vertical(cudaTextureObject_t tmp, uchar4* dst,
                               int w, int h, const float* wts, int r) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int idy = blockIdx.y * blockDim.y + threadIdx.y;
    int offx = blockDim.x * gridDim.x;
    int offy = blockDim.y * gridDim.y;

    for (int i = idy; i < h; i += offy) {
        for (int j = idx; j < w; j += offx) {
            float3 acc = make_float3(0.0f, 0.0f, 0.0f);
            for (int k = -r; k <= r; ++k) {
                float f = wts[k + r];
                float4 p = tmp_tex(tmp, i + k, j); // texture clamp extends the image
    borders
                acc.x += f * p.x;
                acc.y += f * p.y;
                acc.z += f * p.z;
            }
            dst[i * w + j] = make_uchar4(
                (unsigned char)fminf(255.5f, fmaxf(0.0f, acc.x)),
                (unsigned char)fminf(255.5f, fmaxf(0.0f, acc.y)),
                (unsigned char)fminf(255.5f, fmaxf(0.0f, acc.z)), 0);
        }
    }
}

void read_image(const char* path, int& w, int& h, uchar4*& data) {
    FILE* fp = fopen(path, "rb");
    if (!fp) {
        std::fprintf(stderr, "ERROR: cannot open %s\n", path);
        std::exit(0);
    }
    if (fread(&w, sizeof(int), 1, fp) < 1 || fread(&h, sizeof(int), 1, fp) < 1) {
        std::fprintf(stderr, "ERROR: broken image file %s\n", path);
        std::exit(0);
    }
    data = (uchar4*)std::malloc(sizeof(uchar4) * w * h);
    if (fread(data, sizeof(uchar4), (size_t)w * h, fp) < (size_t)w * h) {
        std::fprintf(stderr, "ERROR: broken image file %s\n", path);
        std::exit(0);
```

```
    }
    fclose(fp);
}

void write_image(const char* path, int w, int h, uchar4* data) {
    FILE* fp = fopen(path, "wb");
    fwrite(&w, sizeof(int), 1, fp);
    fwrite(&h, sizeof(int), 1, fp);
    fwrite(data, sizeof(uchar4), (size_t)w * h, fp);
    fclose(fp);
}

// Texture object over a cudaArray with Clamp addressing:
// border pixels are duplicated beyond the image bounds (ip = max(min(i, h), 1)).
void make_object_uchar4(uchar4* host, size_t w, size_t h,
                        cudaArray_t& arr, cudaTextureObject_t& tex) {
    cudaChannelFormatDesc desc = cudaCreateChannelDesc<uchar4>();
    CSC(cudaMallocArray(&arr, &desc, w, h));
    CSC(cudaMemcpy2DToArray(arr, 0, 0, host, sizeof(uchar4) * w,
                            sizeof(uchar4) * w, h, cudaMemcpyHostToDevice));

    cudaResourceDesc rd = {};
    rd.resType = cudaResourceTypeArray;
    rd.res.array.array = arr;

    cudaTextureDesc td = {};
    td.addressMode[0] = cudaAddressModeClamp;
    td.addressMode[1] = cudaAddressModeClamp;
    td.filterMode = cudaFilterModePoint;
    td.normalizedCoords = false;

    CSC(cudaCreateTextureObject(&tex, &rd, &td, nullptr));
}

void make_object_float4(float4* devFrom, size_t w, size_t h,
                        cudaArray_t& arr, cudaTextureObject_t& tex) {
    cudaChannelFormatDesc desc = cudaCreateChannelDesc<float4>();
    CSC(cudaMallocArray(&arr, &desc, w, h));
    CSC(cudaMemcpy2DToArray(arr, 0, 0, devFrom, sizeof(float4) * w,
                            sizeof(float4) * w, h, cudaMemcpyDeviceToDevice));

    cudaResourceDesc rd = {};
    rd.resType = cudaResourceTypeArray;
    rd.res.array.array = arr;

    cudaTextureDesc td = {};
    td.addressMode[0] = cudaAddressModeClamp;
    td.addressMode[1] = cudaAddressModeClamp;
    td.filterMode = cudaFilterModePoint;
    td.normalizedCoords = false;

    CSC(cudaCreateTextureObject(&tex, &rd, &td, nullptr));
}

int main() {
    const int MAX_PATH = 1024;
    char path_in[MAX_PATH], path_out[MAX_PATH];
    if (!fgets(path_in, MAX_PATH, stdin) || !fgets(path_out, MAX_PATH, stdin)) {
        std::fprintf(stderr, "ERROR: expected input and output paths\n");
        return 0;
    }
    // strip newline
    for (char* s : {path_in, path_out}) {
        for (int i = 0; s[i]; ++i) {
            if (s[i] == '\n' || s[i] == '\r') s[i] = '\0';
        }
    }

    int r;
    std::scanf("%d", &r);

    // Grid config for benchmarks (default: 32x32 blocks, 16x16 threads)
    int bx = 32, by = 32, tx = 16, ty = 16;
    if (std::scanf("%d %d", &bx, &by) == 2) {
        std::scanf("%d %d", &tx, &ty);
    }
```

```
    int w, h;
    uchar4* h_src = nullptr;
    read_image(path_in, w, h, h_src);

    // Gaussian kernel weights
    int k = 2 * r + 1;
    float* h_wts = (float*)std::malloc(sizeof(float) * k);
    double sum = 0.0;
    for (int i = 0; i < k; ++i) {
        double x = i - r;
        h_wts[i] = (float)std::exp(-x * x / (2.0 * r * r));
        sum += h_wts[i];
    }
    for (int i = 0; i < k; ++i) h_wts[i] /= (float)sum;

    float* d_wts = nullptr;
    CSC(cudaMalloc(&d_wts, sizeof(float) * k));
    CSC(cudaMemcpy(d_wts, h_wts, sizeof(float) * k, cudaMemcpyHostToDevice));

    float4* d_tmp = nullptr;
    uchar4* d_out = nullptr;
    CSC(cudaMalloc(&d_tmp, sizeof(float4) * w * h));
    CSC(cudaMalloc(&d_out, sizeof(uchar4) * w * h));

    cudaArray_t arr_src = nullptr;
    cudaTextureObject_t tex_src;
    make_object_uchar4(h_src, w, h, arr_src, tex_src);

    const dim3 block(tx, ty);
    dim3 grid(bx, by);

    cudaEvent_t start_all, start_k, stop_k, stop_all;
    CSC(cudaEventCreate(&start_all));
    CSC(cudaEventCreate(&start_k));
    CSC(cudaEventCreate(&stop_k));
    CSC(cudaEventCreate(&stop_all));
    CSC(cudaEventRecord(start_all));

    CSC(cudaEventRecord(start_k));
    gauss_horizontal<<<grid, block>>>(tex_src, d_tmp, w, h, d_wts, r);
    CSC(cudaGetLastError());

    cudaArray_t arr_tmp = nullptr;
    cudaTextureObject_t tex_tmp;
    make_object_float4(d_tmp, w, h, arr_tmp, tex_tmp);
    gauss_vertical<<<grid, block>>>(tex_tmp, d_out, w, h, d_wts, r);
    CSC(cudaGetLastError());
    CSC(cudaEventRecord(stop_k));

    uchar4* h_out = (uchar4*)std::malloc(sizeof(uchar4) * w * h);
    CSC(cudaMemcpy(h_out, d_out, sizeof(uchar4) * w * h, cudaMemcpyDeviceToHost));

    CSC(cudaEventRecord(stop_all));
    CSC(cudaEventSynchronize(stop_all));
    float ms_all, ms_k;
    CSC(cudaEventElapsedTime(&ms_all, start_all, stop_all));
    CSC(cudaEventElapsedTime(&ms_k, start_k, stop_k));
    std::fprintf(stderr, "kernel_ms = %.6f total_ms = %.6f\n", ms_k, ms_all);

    write_image(path_out, w, h, h_out);

    CSC(cudaDestroyTextureObject(tex_src));
    CSC(cudaDestroyTextureObject(tex_tmp));
    CSC(cudaFreeArray(arr_src));
    CSC(cudaFreeArray(arr_tmp));
    CSC(cudaFree(d_wts));
    CSC(cudaFree(d_tmp));
    CSC(cudaFree(d_out));
    std::free(h_src);
    std::free(h_out);
    std::free(h_wts);
    return 0;
}
```

5 Результаты

В соответствии с требованиями выполнены сравнительные замеры производительности на CPU и GPU для различных масштабов изображения ($N = 100\,000$, $5\,000\,000$ и предельный $20\,000\,000$ пикселей, $W \times H = 20000 \times 1000$) для различных конфигураций сетки при фиксированном блоке $\langle 16, 16 \rangle$ нитей и радиусе размытия $r = 100$ (ядро из 201 тапа). Замер CPU выполнен на однопотечной эталонной реализации того же двухпроходного алгоритма (`report/cpu_ref.cpp`). Единицы измерения времени — миллисекунды (мс).

Таблица 1: Сравнение времени работы (мс) и ускорения на CPU и GPU (радиус $r = 100$)

Пикселей N	**Сетка** $\ll \mathbf{B_x B_y} \gg$	**CPU (мс)**	**GPU Ядро (мс)**	**GPU Всего (мс)**	**Ускорение ядра**	**Полное ускорение**
100 000	$\ll 1, 32 \gg$	66.325	1.529	1.975	43.37x	33.59x
100 000	$\ll 1, 256 \gg$	66.325	1.745	2.104	38.01x	31.52x
100 000	$\ll 1, 1024 \gg$	66.325	1.642	2.006	40.40x	33.07x
100 000	$\ll 32, 256 \gg$	66.325	0.901	1.271	73.59x	52.17x
100 000	$\ll 256, 256 \gg$	66.325	0.898	1.219	73.84x	54.42x
100 000	$\ll 1024, 256 \gg$	66.325	1.345	1.687	49.32x	39.33x
100 000	$\ll 1024, 1024 \gg$	66.325	3.542	3.929	18.72x	16.88x
5 000 000	$\ll 1, 32 \gg$	4086.259	14.398	35.076	283.80x	116.50x
5 000 000	$\ll 1, 256 \gg$	4086.259	13.137	27.307	311.04x	149.64x
5 000 000	$\ll 1, 1024 \gg$	4086.259	13.293	33.829	307.40x	120.79x
5 000 000	$\ll 32, 256 \gg$	4086.259	9.745	22.78	419.31x	179.38x
5 000 000	$\ll 256, 256 \gg$	4086.259	9.486	22.165	430.79x	184.36x
5 000 000	$\ll 1024, 256 \gg$	4086.259	9.378	25.868	435.71x	157.96x
5 000 000	$\ll 1024, 1024 \gg$	4086.259	10.725	34.276	381.01x	119.22x
20 000 000	$\ll 1, 32 \gg$	17178.331	152.021	202.017	113.00x	85.03x
20 000 000	$\ll 1, 256 \gg$	17178.331	48.407	105.489	354.87x	162.84x
20 000 000	$\ll 1, 1024 \gg$	17178.331	49.684	138.73	345.75x	123.83x
20 000 000	$\ll 32, 256 \gg$	17178.331	34.142	85.918	503.14x	199.94x
20 000 000	$\ll 256, 256 \gg$	17178.331	33.816	83.659	508.00x	205.34x
20 000 000	$\ll 1024, 256 \gg$	17178.331	33.943	113.664	506.09x	151.13x
20 000 000	$\ll 1024, 1024 \gg$	17178.331	36.293	89.179	473.32x	192.63x

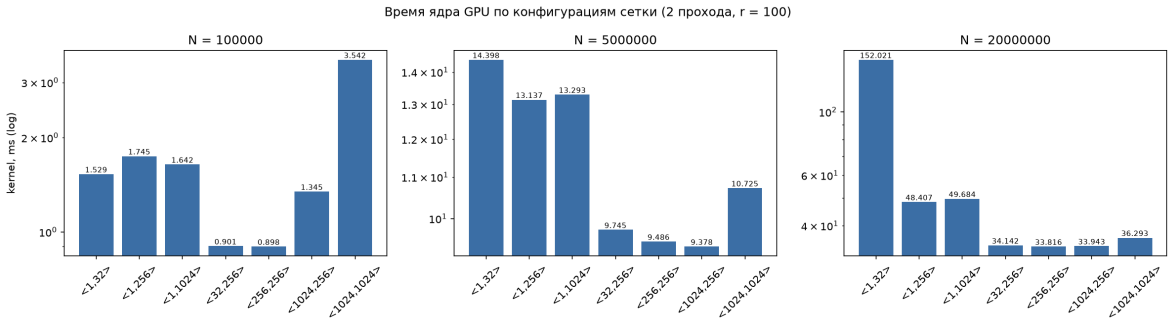


Рис. 1: Зависимость времени выполнения ядра от конфигурации сетки (лог. шкала)

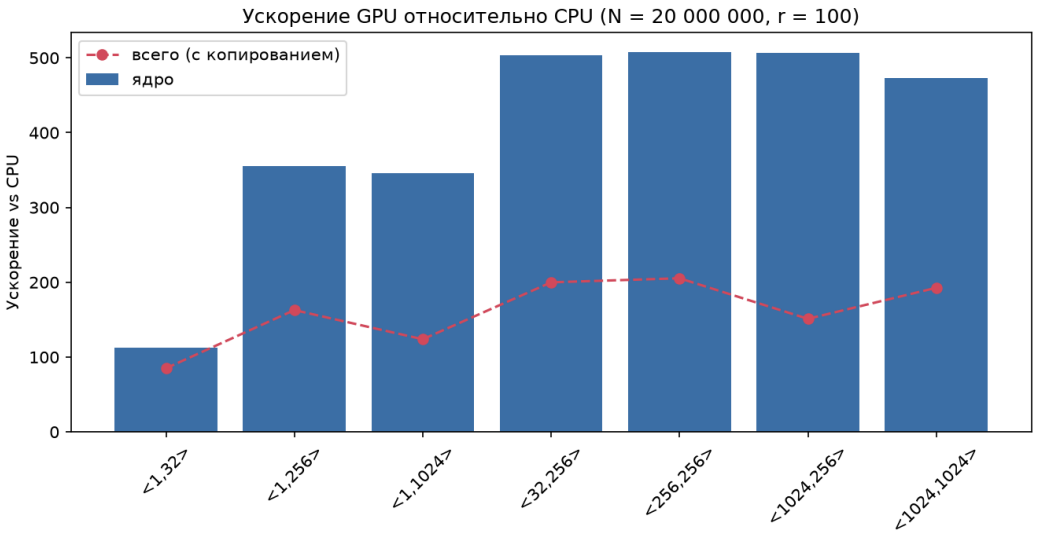


Рис. 2: Ускорение вычислительного ядра GPU относительно CPU ($N = 20\,000\,000$, $r = 100$)

6 Выводы

Область применения алгоритма и типовые задачи

Гауссово размытие — базовая операция обработки изображений, лежащая в основе множества прикладных алгоритмов:

- построение пирамид изображений (mipmap) и высококачественное уменьшение разрешения с предварительной низкочастотной фильтрацией (антиалиасинг);
- компьютерное зрение: сглаживание шума перед детекторами границ (Canny), построение пирамид разности гауссианов (DoG) для локализации ключевых точек (SIFT);
- графика реального времени: постэффект bloom/glow, объемное освещение (light shafts), эффекты дымки и боке (depth of field);
- научные вычисления: ядро сглаживания в симуляциях частиц (SPH), размытие карт плотностей (kernel density estimation).

Оценка сложности программирования и возникшие проблемы

- **Сложность программирования:** алгоритмическая сложность по времени $O(w \cdot h \cdot r)$ (двухпроходная сепарабельная схема), по дополнительной памяти $O(w \cdot h)$ на GPU (промежуточный массив `float4` и текстуры). Двухмерная сетка нитей в сочетании с текстурной памятью в режиме Clamp позволяет переложить граничную логику на аппаратуру и делает ядро компактным.
- **Возникшие проблемы:**
  1. текстурный объект поверх линейной памяти (`cudaResourceTypeLinear`) является одномерным и не поддерживает 2D-индексацию с зажимом по обеим осям — потребовался переход на `cudaResourceTypeArray` с копированием через `cudaMemcpy2DToArray`;

2. точность: при $r = 100$ (201 тап на проход) промежуточные значения необходимо хранить в вещественном виде, поэтому проходы связаны через буфер `float4`, а финальный клампинг значений выполняется в диапазоне $[0, 255]$

Сравнение и объяснение полученных результатов

Эксперименты показали, что:

1. На предельном изображении ($N = 20\,000\,000$ пикселей, $r = 100$) ядро GPU обгоняет однопоточный CPU более чем в 500 раз (33.8 мс против 17 178.3 мс), а полная обработка (включая передачу данных по PCI-Express и текстурные копирования) — более чем в 205 раз.

2. Лучшие конфигурации достигаются при заполнении всех 48 мультипроцессоров ($\ll 256, 256 \gg$, $\ll 32, 256 \gg$): время ядра 33.82 мс против 152.02 мс у конфигурации $\ll 1, 32 \gg$ (ускорение в 4.5 раза при том же алгоритме).

3. Чрезмерно большие сетки ($\ll 1024, 1024 \gg >$ 1 млн нитей) на малых изображениях дают обратный эффект из-за накладных расходов на диспетчеризацию избыточных нитей.